Konsep Dasar Pengujian Perangkat Lunak

Mata Kuliah:
Pengujian Perangkat Lunak (CIF62128)
Disusun oleh:
Software Engineering Lab.

Version: 1.0
Date: 11; Month: February; Year: 2026
© Fakultas Ilmu Komputer, Universitas Brawijaya

filkom.ub.ac.id

Agenda

- Konsep Dasar Pengujian Perangkat Lunak (PL)
- Definisi Pengujian PL
- Terminologi dalam pengujian PL
- Prinsip Pengujian PL
- Klasifikasi Pengujian PL

Capaian Pembelajaran Pertemuan Ini

1. Mahasiswa mampu menjelaskan dengan tepat konsep dasar pengujian perangkat lunak meliputi definisi, tujuan, dan terminologi
2. Mahasiswa mampu menjelaskan dengan tepat 7 prinsip dasar pengujian perangkat lunak beserta penerapannya
3. Mahasiswa mampu membedakan jenis-jenis pengujian berdasarkan berbagai kriteria

© Fakultas Ilmu Komputer, Universitas Brawijaya

Definisi Pengujian Perangkat Lunak

- Pengujian perangkat lunak merupakan disiplin rekayasa yang krusial karena perangkat lunak bersifat **logis** dan tidak akan "aus" seperti perangkat keras, namun setiap kegagalan fungsi menunjukkan adanya kesalahan dalam desain atau proses penerjemahannya menjadi kode (Pressman & Maxim, 2020).
- Dalam RPL modern, **pengujian** bukan lagi sekadar fase akhir sebelum perilisan, melainkan **bagian integral** dari Software Development Life Cycle (SDLC) yang dilakukan **secara paralel sejak tahap analisis kebutuhan** (Sommerville, 2016).
- Fokus utamanya adalah membangun kepercayaan bahwa sistem **layak digunakan** serta meminimalkan risiko operasional (Perry, 2006).

© Fakultas Ilmu Komputer, Universitas Brawijaya

Definisi Pengujian Perangkat Lunak

- Sebuah proses yang mencakup **serangkaian aktivitas** sepanjang siklus hidup pengembangan PL, yang bertujuan untuk **mengevaluasi PL** untuk menentukan apakah PL telah **memenuhi kebutuhan** dan **sesuai** untuk **tujuan** penggunaan yang dimaksudkan (Graham et al., 2012).
- Lebih spesifik lagi, pengujian adalah proses menjalankan program dengan tujuan eksplisit untuk **menemukan kesalahan** atau ***error*** (Pressman & Maxim, 2020).

© Fakultas Ilmu Komputer, Universitas Brawijaya

V&V

Terdapat dua pilar utama dalam pengujian yang dikenal sebagai V&V:

- **Verifikasi**: Menentukan apakah produk kerja (PL) memenuhi spesifikasi kebutuhan yang ditetapkan pada fase RE ("*Apakah kita membangun produk* ***dengan benar***?") (Ammann & Offutt, 2016).
- **Validasi**: Mengevaluasi perangkat lunak pada **akhir** proses pengembangan untuk memastikan kepatuhan terhadap kebutuhan pengguna ("*Apakah kita membangun* ***produk yang benar****?*") (Pressman & Maxim, 2020).

© Fakultas Ilmu Komputer, Universitas Brawijaya

Verifikasi (Verification)

Proses **teknis** untuk memastikan bahwa produk kerja pada setiap fase pengembangan telah memenuhi kebutuhan yang ditetapkan sebelumnya.

- **Spesifikas**i: Jika dokumen spesifikasi menyatakan bahwa "*Lampu di depan layar proyektor harus dapat dikendalikan secara mandiri,*" maka verifikasi dilakukan dengan memeriksa apakah sistem kabel dan sakelar yang dibangun memang mendukung kendali independen tersebut.
- **Inspeksi Kode** (*Code Inspection*): Pengembang melakukan peninjauan baris demi baris pada kode sumber tanpa menjalankannya untuk memastikan tidak ada kesalahan logika, variabel yang tidak terinisialisasi, atau pelanggaran standar pemrograman.
- **Review Kebutuhan**: Tim meninjau dokumen kebutuhan menggunakan daftar periksa (*checklist*) untuk memastikan semua poin kebutuhan tertulis dengan jelas, tidak ambigu, dan dapat diukur secara kuantitatif.
- **Perhitungan Matematis**: Dalam sistem penggajian, verifikasi memastikan bahwa kode program menghitung gaji bruto tepat sesuai dengan rumus yang tertera dalam dokumen desain teknis.

© Fakultas Ilmu Komputer, Universitas Brawijaya

Validasi (Validation)

Proses **evaluasi** pada **akhir** pengembangan untuk memastikan bahwa perangkat lunak memenuhi kebutuhan pengguna dan tujuan bisnisnya.

- **Kepuasan Pengguna**: Meskipun sakelar lampu lulus verifikasi karena bekerja sesuai spesifikasi, validasi dilakukan dengan bertanya **apakah pengguna merasa puas** dengan cara kerja tersebut; jika ternyata tata letak sakelar membingungkan bagi pengguna awam, maka validasi dianggap gagal karena spesifikasi awal tidak mencerminkan kebutuhan nyata pengguna.
- ***User Acceptance Testing*** (UAT): Pelanggan atau pengguna akhir menjalankan perangkat lunak untuk melakukan tugas sehari-hari guna memastikan sistem tersebut benar-benar "layak digunakan" (*fit for purpose*) dalam lingkungan kerja mereka.
- ***Beta Testing***: Perusahaan merilis versi rilis awal perangkat lunak kepada sekelompok kecil pengguna untuk digunakan di lokasi mereka sendiri tanpa campur tangan pengembang, guna melihat apakah produk berfungsi dengan baik dalam kondisi dunia nyata.
- **Pengujian Perjalanan Pengguna (*User Journey*)**: Dalam aplikasi web, validasi dilakukan dengan mensimulasikan seluruh interaksi pengguna dari awal hingga akhir (seperti proses membeli produk hingga pembayaran) untuk memastikan sistem memberikan solusi yang diharapkan pengguna.

© Fakultas Ilmu Komputer, Universitas Brawijaya

V&V

- **Verifikasi** biasanya melibatkan pemeriksaan artefak statis seperti dokumen dan kode melalui inspeksi atau *walkthrough*
- **Validasi** lebih sering melibatkan **eksekusi PL** secara **dinamis** dan keterlibatan langsung pengguna untuk menilai kesesuaian produk dengan tujuan aslinya
- **Tujuan**:
  - ***Defect Prevention***: Mencegah bug masuk tahap *production*
  - ***Quality Assurance***: Memastikan kualitas produk

Key Insight: Verification tanpa Validation = Produk benar secara teknis tapi salah secara

© Fakultas Ilmu Komputer, Universitas Brawijaya

Terminologi dalam Pengujian PL

Rantai penyebab masalah dalam PL menurut Ammann dan Offutt (2016) serta Graham et al. (2012):

1. ***Error*** (Kesalahan/Mistake): Tindakan manusia yang menghasilkan hasil yang tidak akurat, misalnya kesalahan logika saat memprogram (Pressman & Maxim, 2020).
2. ***Fault/Defect*** (Cacat/Bug): Cacat statis di dalam PL atau dokumen spesifikasi yang merupakan manifestasi dari sebuah error (Ammann & Offutt, 2016).
3. ***Failure*** (Kegagalan): Penyimpangan perilaku eksternal perangkat lunak dari apa yang diharapkan oleh pengguna atau spesifikasi saat kode dijalankan (Graham et al., 2012).

© Fakultas Ilmu Komputer, Universitas Brawijaya

Contoh Kasus

- Seorang *programmer* lupa bahwa indeks array di Java dimulai dari 0 dan justru menulis `for (int i = 1; i < x.length; i++)` (ini adalah **Error**).
- Kode tersebut tersimpan di dalam program sebagai **cacat** (ini adalah **Bug**/**Fault**).
- Saat program dijalankan untuk mencari elemen di posisi pertama, sistem melewatkan elemen tersebut dan memberikan hasil yang salah kepada pengguna (ini adalah **Failure**) (Ammann & Offutt, 2016).

© Fakultas Ilmu Komputer, Universitas Brawijaya

Contoh Kasus (ATM)

KEKELIRUAN (ERROR)

Programmer salah memahami logika saldo minimal (Human Action).

CACAT (FAULT)

Seharusnya >=

```
if (saldo > jumlah_tarik)
```

KEGAGALAN (FAILURE)

Pengguna dengan saldo Rp500.000 gagal menarik Rp500.000. Sistem menolak transaksi valid.

Test Oracle

Mekanisme untuk menentukan apakah ***test case*** **PASS** atau **FAIL**

Tipe-tipe Test Oracle:

1. **Specification-based Oracle**
   - Berdasarkan requirements/spec
   - Paling reliable dan umum
2. **Regression Oracle**
   - Membandingkan dengan versi
   - sebelumnya
   - Untuk *regression testing*
3. **Null Oracle**
   - Sistem tidak boleh crash
   - Minimal acceptance criteria
4. **Human Oracle**
   - Manual review oleh expert
   - Untuk usability testing

Oracle Problem:
Sulit menentukan *expected result* untuk complex system

© Fakultas Ilmu Komputer, Universitas Brawijaya

Test Case, Test Suite, dan Test Data

Test Case Components:

ID: TC001
Title: Login Valid User
Precondition: User exists in DB
Test Steps:
1. Navigate to login page
2. Enter valid username
3. Enter valid password
4. Click login button
Expected Result: User logged in

Test Suite:

- Koleksi test cases yang terkait
- Dijalankan bersama untuk tujuan tertentu
- **Contoh**: Login Test Suite, Payment Test
- Suite

Test Data:

- **Input values** untuk test case
- **Valid data**: Memenuhi requirements
- **Invalid data**: Boundary & error conditions
- **Edge cases**: Extreme values

© Fakultas Ilmu Komputer, Universitas Brawijaya

Prinsip Pengujian PL (1/2)

Berdasarkan standar International Software Testing Qualifications Board (ISTQB), terdapat tujuh prinsip utama (Graham et al., 2012):

1. **Pengujian Menunjukkan Adanya Cacat**: Pengujian dapat membuktikan adanya cacat, tetapi tidak dapat membuktikan bahwa perangkat lunak 100% bebas cacat (Aniche, 2022).

2. **Pengujian Melelahkan** (*Exhaustive*) Menguji semua kombinasi input dan prasyarat secara total adalah mustahil. pengujian harus difokuskan pada risiko dan prioritas (Aniche, 2022).

3. ***Early Test***: Aktivitas pengujian harus dimulai sedini mungkin dalam siklus pengembangan untuk mengurangi biaya perbaikan cacat (Sommerville, 2016).

© Fakultas Ilmu Komputer, Universitas Brawijaya

Prinsip Pengujian PL (2/2)

4. ***Defect Clustering***: Sebagian besar cacat biasanya ditemukan pada sejumlah kecil modul inti yang kompleks (Graham et al., 2012).

5. **Paradoks Pestisida:** Jika tes yang sama diulang terus-menerus, tes tersebut tidak akan lagi menemukan cacat baru. Maka, kasus uji harus diperbarui secara berkala (Aniche, 2022).

6. **Pengujian Tergantung Konteks**: Cara menguji sistem e-commerce akan berbeda dengan cara menguji perangkat lunak kendali medis yang kritis (Graham et al., 2012).

7. **Kekeliruan Ketiadaan Kesalahan:** Memperbaiki semua cacat tidak menjamin keberhasilan jika sistem tetap tidak dapat digunakan atau tidak memenuhi kebutuhan pengguna (Graham et al., 2012).

© Fakultas Ilmu Komputer, Universitas Brawijaya

Klasifikasi Pengujian PL (1/4)

A. Berdasarkan Tipe

PENGUJIAN FUNGSIONAL

What the system does.

Menguji fungsi sistem (perhitungan, transaksi) berdasarkan spesifikasi kebutuhan.

PENGUJIAN NON-FUNGSIONAL

How the system works.

Menguji atribut kualitas: Kinerja (*Performance*), Keamanan (*Security*), Kegunaan (*Usability*).

© Fakultas Ilmu Komputer, Universitas Brawijaya

Klasifikasi Pengujian PL (2/4)

B. Berdasarkan Metode Eksekusi

- ***Static Testing***: Pengujian tanpa menjalankan kode, biasanya melalui tinjauan dokumen (reviews) dan inspeksi (Graham et al., 2012).
- ***Dynamic Testing***: Pengujian yang dilakukan dengan menjalankan PL menggunakan data uji (Ammann & Offutt, 2016).

Best Practice: Kombinasi Static + Dynamic testing untuk coverage maksimal

© Fakultas Ilmu Komputer, Universitas Brawijaya

Klasifikasi Pengujian PL (3/4)

C. Berdasarkan Pengetahuan tentang Struktur Internal (Teknik)

- **White-box Testing (*Structural*)**: Pengujian yang didasarkan pada pengetahuan tentang logika internal dan jalur kode program (Pressman & Maxim, 2020).
- **Black-box Testing (*Behavioral*):** Pengujian yang fokus pada fungsionalitas sistem berdasarkan kebutuhan tanpa mengetahui struktur internal kode (Graham et al., 2012).

© Fakultas Ilmu Komputer, Universitas Brawijaya

Klasifikasi Pengujian PL (3/4)

C. Berdasarkan Pengetahuan tentang Struktur Internal (Teknik)

Black-box Testing

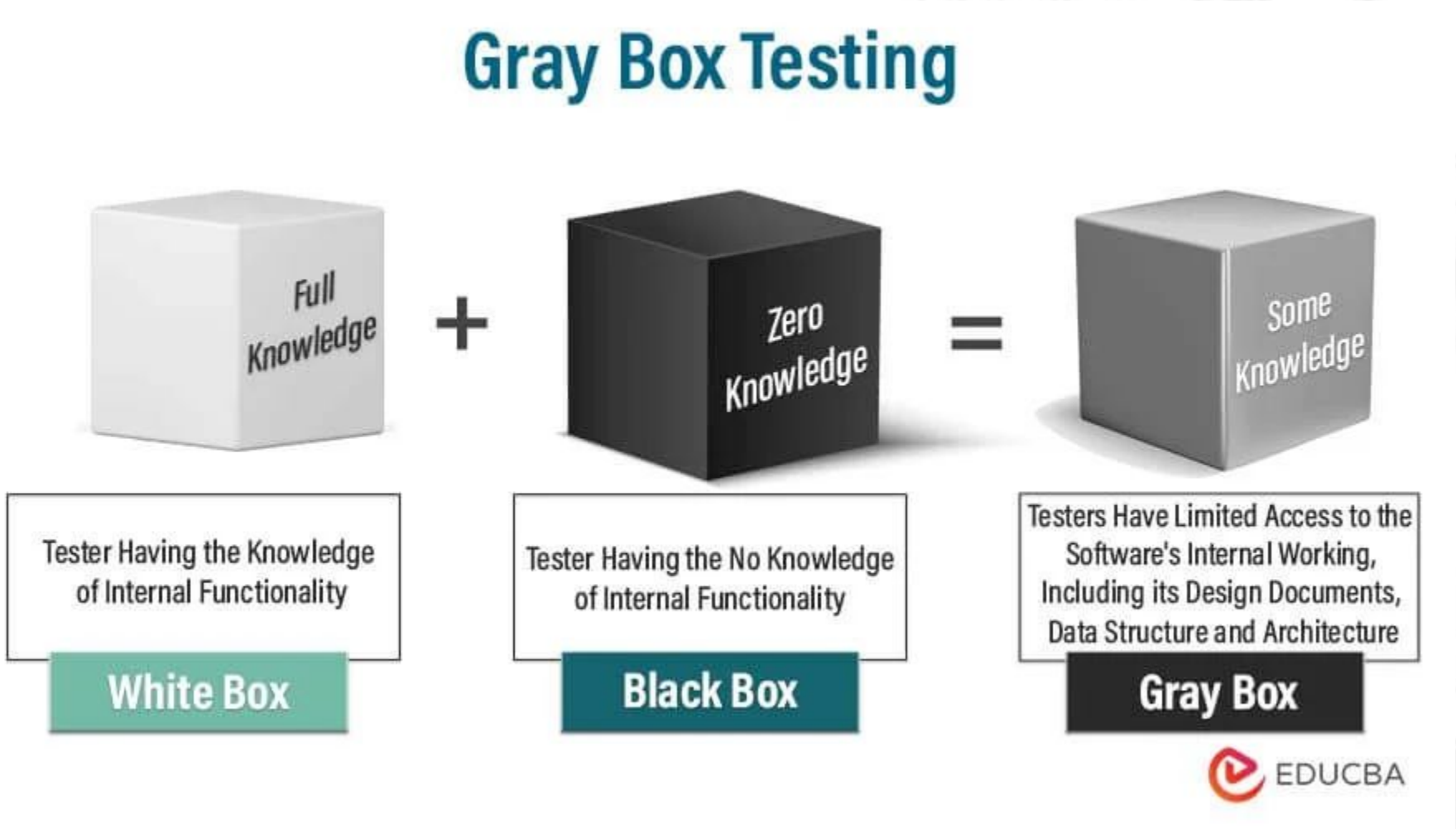
Specification-based. Menguji tanpa melihat struktur kode internal. (Contoh: Boundary Value Analysis).

White-box Testing

Structure-based. Menguji berdasarkan logika internal dan struktur kode. (Contoh: Statement Coverage).

© Fakultas Ilmu Komputer, Universitas Brawijaya

Klasifikasi Pengujian PL (3/4)



Klasifikasi Pengujian PL (4/4)

D. Tingkatan Pengujian (Testing Levels)

Secara hierarki, pengujian dilakukan dari yang terkecil hingga sistem utuh:

1. ***Unit Testing***: Fokus pada komponen terkecil seperti fungsi atau metode (Aniche, 2022).
2. ***Integration Testing***: Menguji interaksi antara beberapa komponen yang telah digabungkan (Pressman & Maxim, 2020).
3. ***System Testing***: Menguji perilaku seluruh sistem secara menyeluruh terhadap kebutuhan fungsional dan non-fungsional (Sommerville, 2016).
4. ***Acceptance Testing***: Pengujian akhir oleh pengguna untuk memutuskan apakah sistem siap dioperasikan (Aniche, 2022).

© Fakultas Ilmu Komputer, Universitas Brawijaya

Test Maturity Models (TMM)

Kerangka kerja atau **standar** yang digunakan untuk menilai dan meningkatkan **kematangan proses pengujian perangkat lunak** dalam sebuah **organisasi.**

- **Level 1: Initial (Awal)**

  Pengujian dilakukan secara ad-hoc, tidak teratur, dan tanpa proses formal. Dokumentasi sangat minim dan pengujian sering kali hanya dianggap sebagai bagian dari fase debugging.

- **Level 2: Managed (Terkelola)**

  Proses dasar sudah mulai diterapkan. Ada rencana pengujian (test plan), strategi pengujian, dan penggunaan metrik dasar untuk memantau kemajuan.

- **Level 3: Defined (Terdefinisi)**

  Pengujian sudah menjadi proses standar yang terintegrasi ke dalam siklus hidup pengembangan perangkat lunak (SDLC). Ada kebijakan organisasi yang jelas dan metodologi pengujian yang konsisten.

- **Level 4: Measured (Terukur)**

  Proses pengujian diukur dan dikendalikan menggunakan metrik statistik untuk mengevaluasi kualitas produk dan efisiensi proses secara objektif.

- **Level 5: Optimized (Dioptimalkan)**

  Fokus pada perbaikan berkelanjutan. Organisasi menggunakan data dari level sebelumnya untuk mencegah cacat (defect prevention) dan terus menyempurnakan proses pengujian secara otomatis dan inovatif.

© Fakultas Ilmu Komputer, Universitas Brawijaya

Test Design Techniques Overview

Specification-Based (Black Box):

- Equivalence Partitioning
- Boundary Value Analysis
- Decision Table Testing
- State Transition Testing
- Use Case Testing

Structure-Based (White Box):

- Statement Coverage
- Branch/Decision Coverage
- Path Coverage

Experience-Based:

- Error Guessing
- Exploratory Testing
- Checklist-Based Testing

Model-Based:

- Classification Tree Method
- Pairwise Testing
- Model Checking

© Fakultas Ilmu Komputer, Universitas Brawijaya

Industry Standards dan Best Practices

Testing Standards:

- ISO/IEC/IEEE 29119: Software Testing
- standard
- IEEE 829: Test Documentation
- ISO 25010: Quality model
- ISTQB: Certification body

Process Models:

- STLC: Software Test Life Cycle
- TPI/TMMi: Test process improvement
- Agile Testing: Agile-specific practices

Best Practices:

- Early and continuous testing
- Risk-based test prioritization
- Test automation pyramid
- Shift-left testing approach
- Continuous integration
- Metrics-driven decisions
- Knowledge sharing culture

© Fakultas Ilmu Komputer, Universitas Brawijaya

Diskusi

Scenario: **Banking Application (Money Transfer Feature)**

Latihan Kelompok (10 menit):

1. Identifikasi **3-5 potensi Kesalahan** yang bisa terjadi dalam pengembangan
2. Jelaskan **Cacat (*defect*)** terkait yang mungkin muncul dalam kode
3. Memprediksi kemungkinan **Kegagalan** yang akan dialami pengguna
4. Menerapkan **2 prinsip ISTQB** pada skenario pengujian
5. Rancangan **1 test case Positif dan 1 Negatif**

Diskusikan hasil per kelompok!

Refleksi Pembelajaran

Check Your Understanding:

1. **Jelaskan perbedaan** antara Error, Defect, dan Failure dengan contoh konkret
2. **Analisis penerapan** 7 prinsip ISTQB dalam proyek software yang kompleks
3. **Evaluasi trade-off** antara Static vs Dynamic testing dalam context tertentu
4. **Berikan contoh** bagaimana Verification dan Validation complement each other

© Fakultas Ilmu Komputer, Universitas Brawijaya

Kesimpulan

Konsep Fundamental yang Harus Dikuasai:

- **Testing Definition**: Evaluasi dan verifikasi untuk memastikan quality
- **Error-Defect-Failure Chain**: Understanding root cause analysis
- **7 ISTQB Principles**: Foundation untuk professional testing
- **Verification vs Validation**: Building right vs right building
- **Static vs Dynamic**: Complementary approaches untuk comprehensive testing
- **Testing Classifications**: Knowledge, execution, dan level perspectives
- **Quality Characteristics**: Functional dan non-functional requirements https://iso25000.com/index.php/en/iso-25000-standards/iso-25010

© Fakultas Ilmu Komputer, Universitas Brawijaya

Referensi

- Ammann, P., & Offutt, J. (2016). Introduction to Software Testing (2nd ed.). Cambridge University Press.
- Aniche, M. (2022). Effective Software Testing: A Developer's Guide. Manning Publications.
- Graham, D., van Veenendaal, E., Evans, I., & Black, R. (2012). Foundations of Software Testing: ISTQB Certification (3rd ed.). Cengage Learning.
- Perry, W. E. (2006). Effective Methods for Software Testing (3rd ed.). Wiley Publishing, Inc.
- Pressman, R. S., & Maxim, B. R. (2020). Software Engineering: A Practitioner's Approach (9th ed.). McGraw-Hill Education.
- Sommerville, I. (2016). Software Engineering (10th ed.). Pearson.

© Fakultas Ilmu Komputer, Universitas Brawijaya